



Frontend Integration Roadmap for Bhumika

Welcome to the Frontend-Backend Integration Guide! This document is prepared to help you seamlessly connect your React/Next.js UI to the AURIX Backend API.

Because we are using a Decoupled Microservice Architecture, you don't need to worry about the AI worker or the database. You will solely interact with my API endpoints using JSON payloads. Here is your exact integration roadmap:



Phase 1: Authentication ("The Door")

Every request to the backend must be authenticated. Since I have configured Supabase Auth in the backend, you will need to handle user logins and pass the Session Token (JWT).

1. **User Login:** You will build the Login/Signup UI. When a user logs in (using the Supabase JS client or standard HTTP requests), Supabase returns an `access_token` (JWT).
2. **Store the Token:** Save this token securely (e.g., in `localStorage` or cookies).
3. **Attach to Headers:** For every API call you make to the backend, you must attach this token in the headers:

```
{  
  
  "Authorization": "Bearer <YOUR_JWT_TOKEN>",  
  
  "Content-Type": "application/json"  
}
```



Phase 2: Triggering a Scan (Ingestion)

When a user pastes a GitHub URL into your Dual-Mode Ingestion Form, you will tell the backend to start a scan.

Endpoint: `POST /api/scans/github`

Headers: `Authorization: Bearer <JWT>`

Body:{

`"github_url": "https://github.com/user/repo",`

`"project_id": "<uuid-of-project>"`

`}`

What you will receive: The backend will immediately queue the job and return a 202 Accepted response.{

`"scan_id": "a1b2c3d4-...",`

`"status": "PENDING",`

`"message": "GitHub scan queued successfully"`

`}`

Save the `scan_id` in your React state—you need it for the next phase!



Phase 3: The Polling Mechanism (Real-Time Status UI)

The AI takes 2-3 minutes to scan the code. While the user waits, you need to show a dynamic progress bar.

Endpoint: `GET /api/scans/<scan_id>`

Headers: `Authorization: Bearer <JWT>`

Your Task: Write a `setInterval` or use SWR/React Query to hit this endpoint every 5 seconds.

While Scanning (Keep Polling):{

```
"scan_id": "a1b2c3d4-...",
```

```
"status": "PENDING"
```

```
}
```

When Finished (Stop Polling!):n

```
{
```

```
"scan_id": "a1b2c3d4-...",
```

```
"status": "COMPLETED",
```

```
"summary": {
```

```
"total_findings": 12,
```

```
"neutralized_count": 0
```

```
},
```

```
"findings": [
```

```
{
```

```
"rule_id": "OWASP-A03",
```

```
"severity": "HIGH",
```

```
"category": "Injection",
```

```
"title": "SQL Injection in Login",
```

```
"file_path": "src/auth.js",
```

```
"line_number": 42,
```

```
"ai_reasoning": "The user input is directly concatenated into the SQL query...",
```

```
"poc_script": "payload = '' OR 1=1 --",
```

```
"patch_code": "db.query('SELECT * FROM users WHERE id = $1', [id])",
```

```
"is_resolved": false
```

```
}
```

```
]
```

```
}
```

🤖 Phase 4: UI Rendering & AI Upgrades

Once you receive the `COMPLETED` response with the massive `findings` array, your core UI work begins:

1. ****Triage Kanban Board****: Feed the ``findings`` array into your Data-grid/Kanban board. Allow the user to filter by ``severity`` and sort by ``category``.
2. ****Vulnerability Detail View****: When a user clicks a bug, use a library like ``react-syntax-highlighter`` to beautifully render the raw ``poc_script`` and ``patch_code`` strings.
3. ****AI Executive Summary Engine (Your AI Upgrade)****: Take the entire JSON response and feed it into a local prompt using LangChain.js or the Vercel AI SDK. Stream a conversational "Threat Landscape Report" using a typewriter effect.
4. ****Contextual AI Chatbot****: Use the ``ai_reasoning`` and ``poc_script`` to seed a slide-out chat window where the user can ask the AI follow-up questions about that specific vulnerability.

Critical Error Handling

If the backend ever returns ``status: "FAILED"``, gracefully stop your polling loop and show a red alert banner to the user (e.g., "Scan failed due to timeout or invalid repository"). ****Do not crash the UI.****